

DSA Mastery Notes

Every Pattern You Need to Crack Product-Based Company Interviews

Java Implementations · Pseudocode · Step-by-Step Visual Traces

Beginner → Advanced

14+ Core Patterns

Java (JDK 8+)

Table of Contents

- P.** Java Prerequisites (Arrays, Strings, Classes, Collections Framework)
- 0.** How to Use These Notes + Big-O Foundations
- 1.** Two Pointers
- 2.** Sliding Window
- 3.** Prefix Sum
- 4.** Frequency Counting & Hashing
- 5.** Matrix Traversal
- 6.** Stack & Monotonic Stack
- 7.** Linked List Patterns
- 8.** Binary Trees & Traversals (Pre/In/Post/Level)
- 9.** Binary Search Trees
- 10.** Heaps & Top-K Elements
- 11.** Graphs: BFS & DFS
- 12.** Backtracking
- 13.** Greedy Algorithms
- 14.** Dynamic Programming (1D, 2D, Knapsack, LCS, etc.)
- 15.** Overlapping Intervals
- 16.** Binary Search on Answer
- 17.** Trie (Prefix Tree)
- 18.** Bit Manipulation
- 19.** Union-Find (Disjoint Set)
- 20.** Final Interview Playbook & Practice Roadmap

P. Java Prerequisites

Before diving into patterns, make sure these Java fundamentals feel automatic — interviewers expect fluent syntax so you can spend your mental energy on the algorithm, not the language.

P.1 Variables & Basic Types

```
int x = 10;                // 32-bit integer
long y = 1000000000000000L; // 64-bit integer (note the L suffix)
double d = 3.14;           // floating point
char c = 'a';              // single character
boolean flag = true;
String name = "Claude";    // String is a class, not a primitive!

// Primitive vs Wrapper classes (needed for generics/collections)
int primitive = 5;
Integer wrapper = 5;       // autoboxed automatically
```

P.2 Arrays — Java's fixed-size, low-level array

Unlike Python lists, Java arrays have a **fixed size** once created. For dynamic resizing, use `ArrayList` (Section P.4).

```
int[] nums = new int[5];           // array of 5 ints, default value 0
int[] nums2 = {5, 2, 9, 1};        // array literal
int[][] grid = new int[3][4];      // 2D array: 3 rows, 4 columns

nums2[0];                          // 5 (access)
nums2.length;                      // 4 (NOT a method -- no parentheses!)
nums2[1] = 100;                    // update

// Common utility methods from java.util.Arrays
import java.util.Arrays;
Arrays.sort(nums2);                // sort ascending, in place
Arrays.toString(nums2);            // "[1, 2, 5, 9]" for printing
Arrays.fill(nums, 7);              // fill entire array with 7
int[] copy = Arrays.copyOf(nums2, nums2.length); // clone
int[] slice = Arrays.copyOfRange(nums2, 1, 3);    // [index 1, index 3)
Arrays.equals(nums, nums2);        // compares contents element-wise
```

Common trap: Comparing arrays with `arr1 == arr2` checks if they are the *same object* in memory, not if their contents match. Always use `Arrays.equals()` for content comparison.

P.3 Strings

Java Strings are immutable, just like Python. Every "modification" method returns a brand new String.

```
String s = "hello world";
s.length();                // 11
s.charAt(0);               // 'h'
s.substring(0, 5);         // "hello" (start inclusive, end exclusive)
s.toUpperCase();           // "HELLO WORLD"
s.toLowerCase();
s.split(" ");              // String[] {"hello", "world"}
s.indexOf("wor");          // 6, or -1 if not found
s.contains("wor");         // true
s.equals(other);           // content comparison (NEVER use == for Strings!)
s.equalsIgnoreCase(other);
s.trim();                  // remove leading/trailing whitespace
s.replace('l', 'L');       // "heLLo worLd"
String.valueOf(123);       // "123" (int -> String)
Integer.parseInt("123");   // 123 (String -> int)
char[] chars = s.toCharArray(); // convert to a char array for in-place work

// Efficient string building (avoid s += ch in a loop -- that's O(n) per append!)
StringBuilder sb = new StringBuilder();
sb.append("hello");
sb.append(' ').append("world");
sb.reverse();              // reverse in place
sb.toString();             // convert back to String when done
```

Critical rule: NEVER compare Strings with `==` — it compares object references, not content. Two equal-looking strings might be different objects. Always use `.equals()`.

P.4 ArrayList — the resizable "array" you'll use constantly

```
import java.util.ArrayList;
import java.util.List;

List<Integer> list = new ArrayList<>();
list.add(5); // append
list.add(0, 100); // insert at index 0
list.get(0); // access -- NOT list[0] like Python!
list.set(0, 99); // update value at index
list.remove(0); // remove BY INDEX (int argument)
list.remove(Integer.valueOf(99)); // remove BY VALUE (must wrap in Integer!)
list.size(); // NOT .length -- that's for arrays only
list.contains(5); // O(n) membership check
list.indexOf(5);
list.isEmpty();
Collections.sort(list); // sort ascending
Collections.reverse(list);
Collections.sort(list, Collections.reverseOrder()); // descending

// Converting between array and ArrayList
Integer[] arr = list.toArray(new Integer[0]);
List<Integer> fromArray = new ArrayList<>(Arrays.asList(1, 2, 3));
```

Common trap: `ArrayList` only works with object types (`Integer`, not `int`). Primitives are auto-boxed/unboxed automatically, but this has a small performance cost — for very large numeric-only data, a plain array is faster.

P.5 The Java Collections Framework Cheat Sheet

Interface	Common Implementation	Use When
<code>List<E></code>	<code>ArrayList</code> , <code>LinkedList</code>	Ordered, indexable, allows duplicates
<code>Set<E></code>	<code>HashSet</code> , <code>TreeSet</code> , <code>LinkedHashSet</code>	Unique elements, fast membership
<code>Map<K, V></code>	<code>HashMap</code> , <code>TreeMap</code> , <code>LinkedHashMap</code>	Key-value pairs, fast lookup
<code>Queue<E></code>	<code>LinkedList</code> , <code>ArrayDeque</code>	FIFO processing, BFS
<code>Deque<E></code>	<code>ArrayDeque</code>	Both-end insert/remove, sliding window, stack or queue
<code>Stack<E></code>	<code>ArrayDeque</code> (preferred) or legacy <code>Stack</code>	LIFO processing
<code>PriorityQueue<E></code>	(itself, backed by a heap)	Top-K, Dijkstra, scheduling

P.6 HashMap, HashSet, and TreeMap in practice

```
import java.util.*;

Map<String, Integer> map = new HashMap<>();
map.put("a", 1);
map.get("a"); // 1
map.getOrDefault("z", 0); // 0 if "z" is missing -- avoids null checks
map.containsKey("a"); // true
map.remove("a");
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
map.put("a", map.getOrDefault("a", 0) + 1); // classic "increment count" pattern

Set<Integer> set = new HashSet<>();
set.add(5);
set.contains(5); // O(1) average
set.remove(5);

TreeMap<Integer, String> sortedMap = new TreeMap<>(); // keys kept in sorted order
sortedMap.firstKey();
```

```
sortedMap.lastKey();
sortedMap.ceilingKey(5);           // smallest key >= 5
sortedMap.floorKey(5);             // largest key <= 5
```

P.7 Deque as Stack AND Queue (preferred over legacy Stack class)

```
import java.util.ArrayDeque;
import java.util.Deque;

Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);           // add to front
stack.pop();             // remove from front -- LIFO behavior

Deque<Integer> queue = new ArrayDeque<>();
queue.offer(1);          // add to back
queue.poll();            // remove from front -- FIFO behavior

Deque<Integer> window = new ArrayDeque<>(); // used as a monotonic deque
window.addLast(5);
window.addFirst(5);
window.removeLast();
window.removeFirst();
window.peekFirst();
window.peekLast();
```

P.8 PriorityQueue (Java's built-in heap)

```
import java.util.PriorityQueue;
import java.util.Collections;

PriorityQueue<Integer> minHeap = new PriorityQueue<>();           // min-heap by default
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // max-heap

minHeap.offer(5);
minHeap.peek();         // smallest element, O(1)
minHeap.poll();         // remove & return smallest, O(log n)

// Custom comparator -- e.g., min-heap of int[] pairs sorted by 2nd element
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
```

P.9 Loops, Conditionals & Common Syntax

```
for (int i = 0; i < 5; i++) { }           // classic for loop
for (int num : nums) { }                 // enhanced for-each loop

int i = 0;
while (i < 5) { i++; }
do { i++; } while (i < 10);

if (x > 0) { }
else if (x == 0) { }
else { }

switch (x) {
    case 1: System.out.println("one"); break;
    default: System.out.println("other");
}

// Ternary operator
int max = (a > b) ? a : b;
```

P.10 Methods (Functions)

```
public static int add(int a, int b) {
    return a + b;
}

// Method overloading -- same name, different parameter types
public static double add(double a, double b) {
    return a + b;
}
```

P.11 Classes & Object-Oriented Basics

```
class Animal {
    static int speciesCount = 0;    // STATIC field, shared by all instances
    String name;                    // INSTANCE field, unique per object
    String sound;

    Animal(String name, String sound) {    // constructor
        this.name = name;
        this.sound = sound;
        speciesCount++;
    }

    String makeSound() {                // instance method
        return name + " says " + sound;
    }

    @Override
    public String toString() {           // controls what prints for this object
        return "Animal(" + name + ")";
    }

    @Override
    public boolean equals(Object o) {     // controls how .equals() compares objects
        if (!(o instanceof Animal)) return false;
        return this.name.equals(((Animal) o).name);
    }

    @Override
    public int hashCode() {               // required if you override equals()
        return name.hashCode();
    }
}

class Dog extends Animal {              // INHERITANCE
    Dog(String name) {
        super(name, "Woof");            // call the parent constructor
    }

    String fetch() {
        return name + " fetches the ball!";
    }
}

Dog d = new Dog("Rex");
System.out.println(d.makeSound());      // "Rex says Woof" (inherited)
System.out.println(d.fetch());           // "Rex fetches the ball!"
```

Why this matters for DSA: Every linked-list/tree problem starts with a tiny class like `ListNode` or `TreeNode` (shown in the sections ahead). Custom objects need `Comparable<T>` implemented (the `compareTo` method) if you want to sort them or use them in a `PriorityQueue` without a custom comparator, and `equals()`/`hashCode()` overridden if you want to use them as `HashMap` keys or in a `HashSet`.

P.12 Generics (why you see <T> everywhere)

```
// Generics let one class/method work with any type safely
class Box<T> {
    private T item;
    void set(T item) { this.item = item; }
    T get() { return item; }
}

Box<String> stringBox = new Box<>();
stringBox.set("hello");

// Generic method
public static <T> void printAll(List<T> list) {
    for (T item : list) System.out.println(item);
}
```

P.13 Recursion Basics (used constantly in Trees, Backtracking, DP)

```
public static int factorial(int n) {
    if (n <= 1) return 1;           // BASE CASE
    return n * factorial(n - 1);    // RECURSIVE CASE
}
```

P.14 Java vs Python Quick-Reference for DSA

Concept	Python	Java
Dynamic array	<code>list</code>	<code>ArrayList<></code>
Get length	<code>len(x)</code>	<code>arr.length</code> (array) / <code>list.size()</code> (ArrayList)
Hash map	<code>dict</code>	<code>HashMap<></code>
Hash set	<code>set</code>	<code>HashSet<></code>
Stack	<code>list</code> (append/pop)	<code>ArrayDeque<></code> (push/pop)
Queue	<code>collections.deque</code>	<code>ArrayDeque<></code> (offer/poll)
Heap	<code>heapq</code> module	<code>PriorityQueue<></code>
String building	join a list	<code>StringBuilder</code>
Infinity	<code>float('inf')</code>	<code>Integer.MAX_VALUE</code> / <code>Double.POSITIVE_INFINITY</code>

0. How to Use These Notes + Big-O Foundations

0.1 How to study this document

Every pattern below follows the same structure:

1. **What it is** — the core idea in plain English.
2. **When to use it** — signal words that hint at the pattern.
3. **Pseudocode** — language-agnostic steps.
4. **Java implementation** — a clean, interview-ready template.
5. **Step-by-step visual trace** — a worked example.
6. **Complexity** — time and space.
7. **Practice problems** — easy → hard.

Golden rule: Don't memorize code. Memorize the *decision process*: "sorted array, need pairs → Two Pointers." "subarray/substring with a condition → Sliding Window." Patterns are shortcuts for recognizing the right algorithm quickly.

0.2 What is Big-O Notation?

Big-O describes how running time or memory grows as input size (n) grows, ignoring constants and lower-order terms.

	$n = 1$	$n = 10$	$n = 1000$	
$O(1)$	1	1	1	(constant)
$O(\log n)$	1	3	10	(grows very slowly)
$O(n)$	1	10	1000	(linear)
$O(n \log n)$	1	33	~9966	(linearithmic)
$O(n^2)$	1	100	1,000,000	(quadratic)
$O(2^n)$	2	1024	1.07×10^{301}	(exponential - avoid!)

Complexity	Name	Typical Pattern
$O(1)$	Constant	Array index access, hash map lookup
$O(\log n)$	Logarithmic	Binary search, balanced BST operations
$O(n)$	Linear	Single loop over input
$O(n \log n)$	Linearithmic	Efficient sorting (merge/heap sort)
$O(n^2)$	Quadratic	Nested loops, brute-force pairs
$O(2^n)$	Exponential	Brute-force subsets
$O(n!)$	Factorial	Brute-force permutations

0.3 Core Data Structures Cheat Sheet (Java classes)

Structure	Java Class	Access	Search	Insert	Delete
Array	<code>int[]</code>	$O(1)$	$O(n)$	-	-
Dynamic Array	<code>ArrayList<></code>	$O(1)$	$O(n)$	$O(1)$ amortized end	$O(n)$
Linked List	<code>LinkedList<></code>	$O(n)$	$O(n)$	$O(1)$ at known node	$O(1)$ at known node
Hash Map/Set	<code>HashMap<></code> / <code>HashSet<></code>	-	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
Stack	<code>ArrayDeque<></code>	$O(n)$	$O(n)$	$O(1)$ top	$O(1)$ top
Queue/Deque	<code>ArrayDeque<></code>	$O(n)$	$O(n)$	$O(1)$ ends	$O(1)$ ends
Binary Heap	<code>PriorityQueue<></code>	$O(1)$ top	$O(n)$	$O(\log n)$	$O(\log n)$
BST	<code>TreeMap<></code> / <code>TreeSet<></code>	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg
Graph (adj list)	<code>Map<Integer, List<Integer>></code>	-	$O(V+E)$	$O(1)$	$O(E)$

Interview tip: Always state the brute-force approach first, then say "we can optimize using X pattern." Interviewers grade your thinking process, not just the final code.

1. Two Pointers

1.1 What it is

Use two index variables that move through an array/string — usually sorted — to avoid a nested loop. Instead of checking every pair ($O(n^2)$), the pointers move intelligently based on a comparison, giving $O(n)$.

Signal words: "sorted array", "pair that sums to X", "palindrome", "remove duplicates in-place", "container with most water", "merge two sorted lists".

1.2 Two Common Shapes

Shape	Description	Example
Opposite ends	left starts at 0, right starts at n-1, move toward each other	Two Sum (sorted), palindrome check, container with most water
Same direction (fast/slow)	both start at 0, move at different speeds	Remove duplicates, cycle detection

1.3 Pseudocode (opposite ends)

```
left = 0
right = length(array) - 1
while left < right:
    if condition_met(array[left], array[right]):
        record answer
        move both pointers
    elif array[left] + array[right] < target:
        left += 1      # need a bigger sum
    else:
        right -= 1     # need a smaller sum
```

1.4 Java Implementation — Two Sum on Sorted Array

```
public static int[] twoSumSorted(int[] nums, int target) {
    // Time: O(n)   Space: O(1)
    int left = 0, right = nums.length - 1;

    while (left < right) {
        int currentSum = nums[left] + nums[right];
        if (currentSum == target) {
            return new int[]{left, right};
        } else if (currentSum < target) {
            left++;
            // sum too small -> increase it
        } else {
            right--;
            // sum too big -> decrease it
        }
    }
    return new int[]{-1, -1}; // no pair found
}
```

1.5 Step-by-Step Visual Trace

Array = [2, 7, 11, 15] , target = 18

Step 1:	[2, 7, 11, 15]	left=0 (2)	right=3 (15)	sum=17 < 18 -> left++
		L	R	
Step 2:	[2, 7, 11, 15]	left=1 (7)	right=3 (15)	sum=22 > 18 -> right--
		L	R	
Step 3:	[2, 7, 11, 15]	left=1 (7)	right=2 (11)	sum=18 == 18 -> FOUND! [1,2]
		L	R	

1.6 Java Implementation — Valid Palindrome (opposite ends)

```
public static boolean isPalindrome(String s) {
    StringBuilder cleaned = new StringBuilder();
    for (char ch : s.toCharArray()) {
        if (Character.isLetterOrDigit(ch)) {
            cleaned.append(Character.toLowerCase(ch));
        }
    }
    int left = 0, right = cleaned.length() - 1;
    while (left < right) {
        if (cleaned.charAt(left) != cleaned.charAt(right)) return false;
        left++;
        right--;
    }
    return true;
}
```

1.7 Java Implementation — Remove Duplicates (fast/slow, same direction)

```
public static int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int slow = 0;
    for (int fast = 1; fast < nums.length; fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;    // new length
}
```

```
nums = [1, 1, 2, 2, 3]
slow=0 fast=1: nums[1]==nums[0] -> skip
slow=0 fast=2: nums[2]!=nums[0] -> slow=1, nums[1]=2 => [1,2,2,2,3]
slow=1 fast=3: nums[3]==nums[1] -> skip
slow=1 fast=4: nums[4]!=nums[1] -> slow=2, nums[2]=3 => [1,2,3,2,3]
Result length = 3 -> first 3 elements [1,2,3] are the unique sorted values
```

1.8 Complexity

Time: $O(n)$ — each pointer traverses the array at most once.

Space: $O(1)$ — only a couple of index variables.

1.9 Practice Problems (Easy → Hard)

Easy: Valid Palindrome · Two Sum II (sorted input) · Remove Duplicates from Sorted Array · Move Zeroes

Medium: 3Sum · Container With Most Water · Sort Colors (Dutch National Flag) · Trapping Rain Water (two-pointer variant)

Hard: 4Sum · Minimum Window covering all characters (hybrid with sliding window) · Merge two sorted arrays in-place from the end

2. Sliding Window

2.1 What it is

A "window" is a contiguous range `[left, right]` inside an array/string. We slide it: expand the right edge to include new elements and shrink the left edge to remove old ones, updating a running result incrementally instead of recomputing from scratch.

Signal words: "contiguous subarray/substring", "longest/shortest/maximum/minimum window", "at most K distinct", "sum equals/at least/at most K".

2.2 Two Flavors

Type	When	Example
Fixed-size window	Window size K is given	Max sum subarray of size K
Variable-size window	Grow/shrink based on a condition	Longest substring without repeating characters

2.3 Pseudocode (variable window)

```
left = 0
window_state = {}
for right in range(n):
    add array[right] to window_state
    while window_is_invalid(window_state):
        remove array[left] from window_state
        left += 1
    update_answer(window_state, left, right)
```

2.4 Java Implementation — Longest Substring Without Repeating Characters

```
public static int lengthOfLongestSubstring(String s) {
    // Time: O(n)   Space: O(min(n, charset))
    Set<Character> seen = new HashSet<>();
    int left = 0, best = 0;

    for (int right = 0; right < s.length(); right++) {
        while (seen.contains(s.charAt(right))) {
            seen.remove(s.charAt(left));
            left++;
        }
        seen.add(s.charAt(right));
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

2.5 Step-by-Step Visual Trace

S = "abcabcbb"

right=0 'a':	window="a"	seen={a}	best=1
right=1 'b':	window="ab"	seen={a,b}	best=2
right=2 'c':	window="abc"	seen={a,b,c}	best=3
right=3 'a':	DUPLICATE 'a' -> remove s[left]='a', left=1		
	window="bca"	seen={b,c,a}	best=3
right=4 'b':	DUPLICATE 'b' -> remove s[left]='b', left=2		
	window="cab"	seen={c,a,b}	best=3
right=5 'c':	DUPLICATE 'c' -> remove s[left]='c', left=3		
	window="abc"	seen={a,b,c}	best=3
right=6 'b':	DUPLICATE 'b' -> remove s[left]='a', left=4; remove s[left]='b', left=5		
	window="cb"	seen={c,b}	best=3
right=7 'b':	DUPLICATE 'b' -> remove s[left]='c', left=6; window="b" seen={b}		best=3

2.6 Java Implementation — Fixed Size Window (Max Sum of Size K)

```
public static int maxSumSubarray(int[] nums, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += nums[i];
    int best = windowSum;

    for (int right = k; right < nums.length; right++) {
        windowSum += nums[right] - nums[right - k];    // add new, drop oldest
        best = Math.max(best, windowSum);
    }
    return best;
}
```

2.7 Java Implementation — Minimum Window Substring (Hard, classic)

```
public static String minWindow(String s, String t) {
    if (s.isEmpty() || t.isEmpty()) return "";

    Map<Character, Integer> need = new HashMap<>();
    for (char ch : t.toCharArray()) need.put(ch, need.getOrDefault(ch, 0) + 1);

    int missing = t.length();
    int left = 0, bestLeft = 0, bestLen = Integer.MAX_VALUE;

    for (int right = 0; right < s.length(); right++) {
        char ch = s.charAt(right);
        if (need.getOrDefault(ch, 0) > 0) missing--;
        need.put(ch, need.getOrDefault(ch, 0) + 1);

        while (missing == 0) {
            if (right - left + 1 < bestLen) {
                bestLeft = left;
                bestLen = right - left + 1;
            }
            char leftChar = s.charAt(left);
            need.put(leftChar, need.get(leftChar) - 1);
            if (need.get(leftChar) > 0) missing++;
            left++;
        }
    }
    return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestLeft, bestLeft + bestLen);
}
```

2.8 Complexity

Time: $O(n)$ — left and right pointers each move forward at most n times.

Space: $O(k)$ where k is the size of the character set / map being tracked.

2.9 Practice Problems (Easy → Hard)

Easy: Maximum Sum Subarray of Size K · Average of Subarrays of Size K

Medium: Longest Substring Without Repeating Characters · Longest Repeating Character Replacement · Permutation in String · Fruit Into Baskets

Hard: Minimum Window Substring · Sliding Window Maximum · Substring with Concatenation of All Words

3. Prefix Sum

3.1 What it is

A prefix sum array stores the running total up to each index. Once built, the sum of any subarray `[i, j]` can be found in $O(1)$ instead of re-adding elements every time.

Signal words: "range sum query", "subarray sum equals K", "multiple queries about sums of ranges".

3.2 The Core Formula

```
prefix[0] = 0
prefix[i] = prefix[i-1] + nums[i-1]

sum(nums[i..j]) = prefix[j+1] - prefix[i]
```

3.3 Java Implementation — Range Sum Query

```
class PrefixSum {
    private int[] prefix;

    public PrefixSum(int[] nums) {
        prefix = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }

    public int rangeSum(int i, int j) { // sum of nums[i..j] inclusive, O(1) per query
        return prefix[j + 1] - prefix[i];
    }
}

// Example
int[] nums = {2, 4, 5, 1, 3};
PrefixSum ps = new PrefixSum(nums);
System.out.println(ps.rangeSum(1, 3)); // 4 + 5 + 1 = 10
```

3.4 Step-by-Step Visual Trace

```
nums:    [ 2,  4,  5,  1,  3]
index:    0   1   2   3   4

prefix: [0, 2, 6, 11, 12, 15]
index:  0  1  2  3  4  5
```

```
range_sum(1,3) = prefix[4] - prefix[1] = 12 - 2 = 10 (matches 4+5+1 = 10)
```

3.5 Java Implementation — Subarray Sum Equals K (prefix sum + hashmap)

```
public static int subarraySum(int[] nums, int k) {
    // Time: O(n)    Space: O(n)
    Map<Integer, Integer> countMap = new HashMap<>();
    countMap.put(0, 1); // empty prefix sums to 0
    int runningSum = 0, result = 0;

    for (int num : nums) {
        runningSum += num;
        result += countMap.getOrDefault(runningSum - k, 0);
        countMap.put(runningSum, countMap.getOrDefault(runningSum, 0) + 1);
    }
    return result;
}
```

3.6 2D Prefix Sum (Matrix Range Sum)

```
public static int[][] build2DPrefix(int[][] matrix) {
    int rows = matrix.length, cols = matrix[0].length;
    int[][] prefix = new int[rows + 1][cols + 1];
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            prefix[r+1][c+1] = matrix[r][c] + prefix[r][c+1] + prefix[r+1][c] - prefix[r][c];
        }
    }
    return prefix;
}

public static int regionSum(int[][] prefix, int r1, int c1, int r2, int c2) {
    return prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1];
}
```

3.7 Complexity

Build: $O(n)$ time, $O(n)$ space.

Each query: $O(1)$ time after the prefix array is built.

3.8 Practice Problems (Easy → Hard)

Easy: Running Sum of 1D Array · Range Sum Query - Immutable

Medium: Subarray Sum Equals K · Product of Array Except Self · Contiguous Array

Hard: Range Sum Query 2D - Immutable · Maximum Size Subarray Sum Equals K · Count of Range Sum

4. Frequency Counting & Hashing

4.1 What it is

Use a `HashMap` or `HashSet` to record how many times something appears, or whether it has been seen, in $O(1)$ average time per lookup.

Signal words: "anagram", "duplicate", "frequency", "count occurrences", "group by", "have I seen this before".

4.2 Java Implementation — Valid Anagram

```
public static boolean isAnagram(String s, String t) {
    // Time: O(n)    Space: O(1) -- at most 26 lowercase letters
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (char ch : s.toCharArray()) count[ch - 'a']++;
    for (char ch : t.toCharArray()) count[ch - 'a']--;
    for (int c : count) if (c != 0) return false;
    return true;
}
```

4.3 Java Implementation — Two Sum (unsorted array, using HashMap)

```
public static int[] twoSum(int[] nums, int target) {
    // Time: O(n)    Space: O(n)
    Map<Integer, Integer> seen = new HashMap<>(); // value -> index
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            return new int[] {seen.get(complement), i};
        }
        seen.put(nums[i], i);
    }
    return new int[] {};
}
```

4.4 Step-by-Step Visual Trace

nums = [2, 7, 11, 15], target = 9

i=0 num=2	complement=7	seen={}	-> not found, add seen={2:0}
i=1 num=7	complement=2	seen={2:0}	-> FOUND 2 at index 0! return [0, 1]

4.5 Java Implementation — Group Anagrams

```
public static List<List<String>> groupAnagrams(String[] strs) {
    // Time: O(n * k log k)    Space: O(n * k)
    Map<String, List<String>> groups = new HashMap<>();
    for (String word : strs) {
        char[] chars = word.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);
        groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
    }
    return new ArrayList<>(groups.values());
}
```

4.6 Java Implementation — First Non-Repeating Character

```
public static int firstUniqChar(String s) {
    Map<Character, Integer> freq = new HashMap<>();
    for (char ch : s.toCharArray()) freq.put(ch, freq.getOrDefault(ch, 0) + 1);
    for (int i = 0; i < s.length(); i++) {
```

```
        if (freq.get(s.charAt(i)) == 1) return i;
    }
    return -1;
}
```

4.7 HashMap vs HashSet Cheat Sheet

Use a HashSet when	Use a HashMap when
You only need "have I seen this?"	You need to track a count or map a key to extra info
Detecting duplicates	Frequency counting, memoization, grouping
Fast membership for graph visited nodes	Two Sum style "value -> index" lookups

4.8 Complexity

Time: $O(1)$ average per insert/lookup, $O(n)$ to build over the whole input.

Space: $O(n)$ worst case for the map/set.

4.9 Practice Problems (Easy → Hard)

Easy: Two Sum · Valid Anagram · Contains Duplicate · First Unique Character in a String

Medium: Group Anagrams · Top K Frequent Elements · Longest Consecutive Sequence · Subarray Sum Equals K

Hard: LRU Cache (HashMap + doubly linked list) · Substring with Concatenation of All Words · Design Twitter

5. Matrix Traversal

5.1 What it is

A 2D grid problem, treated as a graph where each cell is a node connected to its (usually 4) neighbors. Traversal patterns include row/column scanning, spiral order, and flood-fill (BFS/DFS on a grid).

Signal words: "grid", "matrix", "islands", "spiral", "rotate image", "flood fill", "shortest path in a grid".

5.2 The 4-Direction Template

```
int[][] directions = {{-1,0}, {1,0}, {0,-1}, {0,1}}; // up, down, left, right

boolean inBounds(int r, int c, int rows, int cols) {
    return r >= 0 && r < rows && c >= 0 && c < cols;
}
```

5.3 Java Implementation — Number of Islands (DFS flood fill)

```
public static int numIslands(char[][] grid) {
    // Time: O(rows * cols) Space: O(rows * cols) worst case recursion stack
    if (grid == null || grid.length == 0) return 0;
    int rows = grid.length, cols = grid[0].length;
    int islands = 0;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1') {
                islands++;
                dfs(grid, r, c, rows, cols);
            }
        }
    }
    return islands;
}

private static void dfs(char[][] grid, int r, int c, int rows, int cols) {
    if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] != '1') return;
    grid[r][c] = '0'; // mark visited by sinking it
    dfs(grid, r + 1, c, rows, cols);
    dfs(grid, r - 1, c, rows, cols);
    dfs(grid, r, c + 1, rows, cols);
    dfs(grid, r, c - 1, rows, cols);
}
```

5.4 Step-by-Step Visual Trace

```
Grid:
1 1 0 0
    1 1 0 0
      0 0 1 0
0 0 0 1      1 1 0 0  --> 0 0 0 0  (all connected 1s -> 0)
                          0 0 1 0
                          0 0 0 1

Start scanning left-to-right, top-to-bottom.
Find land at (0,0) -> island #1 -> DFS sinks
the whole connected block:
1 1 0 0      0 0 0 0
0 0 1 0      0 0 1 0
0 0 0 1      0 0 0 1

Continue scanning: find land at (2,2) -> island #2 -> sink it.
Continue scanning: find land at (3,3) -> island #3 -> sink it.
Total islands = 3
```

5.5 Java Implementation — Rotate Image 90 Degrees (in-place)

```
public static void rotate(int[][] matrix) {
    // Time: O(n^2) Space: O(1) in place
    int n = matrix.length;
```

```

for (int i = 0; i < n; i++) { // Step 1: transpose
    for (int j = i + 1; j < n; j++) {
        int temp = matrix[i][j];
        matrix[i][j] = matrix[j][i];
        matrix[j][i] = temp;
    }
}
for (int[] row : matrix) { // Step 2: reverse each row
    for (int left = 0, right = row.length - 1; left < right; left++, right--) {
        int temp = row[left];
        row[left] = row[right];
        row[right] = temp;
    }
}
}

```

5.6 Java Implementation — Spiral Matrix Traversal

```

public static List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> result = new ArrayList<>();
    if (matrix.length == 0) return result;

    int top = 0, bottom = matrix.length - 1;
    int left = 0, right = matrix[0].length - 1;

    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; c++) result.add(matrix[top][c]);
        top++;
        for (int r = top; r <= bottom; r++) result.add(matrix[r][right]);
        right--;
        if (top <= bottom) {
            for (int c = right; c >= left; c--) result.add(matrix[bottom][c]);
            bottom--;
        }
        if (left <= right) {
            for (int r = bottom; r >= top; r--) result.add(matrix[r][left]);
            left++;
        }
    }
    return result;
}

```

1 2 3	Spiral order visits: 1 -> 2 -> 3 -> 6 -> 9 -> 8 -> 7 -> 4 -> 5
4 5 6	Shrinking boundary each lap: top++, right--, bottom--, left++
	7 8 9

5.7 Complexity

Time: $O(\text{rows} \times \text{cols})$ — every cell visited a constant number of times.

Space: $O(\text{rows} \times \text{cols})$ worst case for DFS recursion stack or BFS queue; $O(1)$ for simple scans/rotations.

5.8 Practice Problems (Easy → Hard)

Easy: Transpose Matrix · Flood Fill · Toeplitz Matrix

Medium: Number of Islands · Rotate Image · Spiral Matrix · Set Matrix Zeroes · Word Search

Hard: Max Area of Island · Shortest Path in Binary Matrix (BFS) · Dungeon Game · Making A Large Island

6. Stack & Monotonic Stack

6.1 What a stack is

LIFO. In Java, use `ArrayDeque` as a stack (preferred over the legacy `Stack` class): `push()` to add, `pop()` to remove the top.

Signal words: "matching brackets/parentheses", "next greater/smaller element", "undo operation", "evaluate expression", "largest rectangle in histogram".

6.2 Java Implementation — Valid Parentheses

```
public static boolean isValid(String s) {
    // Time: O(n)    Space: O(n) worst case
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Character> pairs = Map.of(')', '(', ']', '['), '}', '{');

    for (char ch : s.toCharArray()) {
        if (ch == '(' || ch == '[' || ch == '{') {
            stack.push(ch);
        } else {
            if (stack.isEmpty() || stack.pop() != pairs.get(ch)) return false;
        }
    }
    return stack.isEmpty();
}
```

6.3 What is a Monotonic Stack?

A monotonic stack keeps its elements always increasing (or decreasing) from bottom to top. When a new element breaks that order, we pop elements off until order is restored — and while popping, we learn something useful (like "next greater element" for the popped item).

6.4 Java Implementation — Next Greater Element

```
public static int[] nextGreaterElements(int[] nums) {
    // Time: O(n) -- each index pushed & popped at most once
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>(); // stores INDICES

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && nums[i] > nums[stack.peek()]) {
            int idx = stack.pop();
            result[idx] = nums[i];
        }
        stack.push(i);
    }
    return result;
}
```

6.5 Step-by-Step Visual Trace

nums = [2, 1, 2, 4, 3]

i=0 val=2	stack=[]	-> push 0	stack=[0]
i=1 val=1	1 < nums[0]=2	-> push 1	stack=[0,1]
i=2 val=2	2 > nums[1]=1	-> pop 1, result[1]=2	
	2 == nums[0]=2 (not strictly greater)	-> push 2	stack=[0,2]
i=3 val=4	4 > nums[2]=2	-> pop 2, result[2]=4	
	4 > nums[0]=2	-> pop 0, result[0]=4	
	stack empty	-> push 3	stack=[3]
i=4 val=3	3 < nums[3]=4	-> push 4	stack=[3,4]

End: indices left in stack [3,4] get result = -1 (no next greater)

6.6 Java Implementation — Largest Rectangle in Histogram (Hard classic)

```
public static int largestRectangleArea(int[] heights) {
    // Time: O(n)    Space: O(n)
    Deque<Integer> stack = new ArrayDeque<>();    // indices, increasing heights bottom->top
    int maxArea = 0;
    int n = heights.length;

    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];        // sentinel to flush remaining stack
        while (!stack.isEmpty() && heights[stack.peek()] > h) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
```

6.7 Complexity

Time: $O(n)$ for monotonic stack problems — amortized, since each element is pushed once and popped at most once.

Space: $O(n)$ for the stack.

6.8 Practice Problems (Easy → Hard)

Easy: Valid Parentheses · Baseball Game · Min Stack

Medium: Daily Temperatures · Next Greater Element I/II · Evaluate Reverse Polish Notation · Online Stock Span

Hard: Largest Rectangle in Histogram · Trapping Rain Water · Maximal Rectangle · Remove Duplicate Letters

7. Linked List Patterns

7.1 What it is

A chain of nodes where each node points to the next. Key tricks: **dummy head node** (simplifies edge cases), and **fast/slow pointers** (find middle, detect cycles).

Signal words: "reverse a linked list", "detect a cycle", "middle of the list", "merge two lists", "remove Nth node from end".

7.2 Node Definition

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
    ListNode(int val, ListNode next) { this.val = val; this.next = next; }
}
```

7.3 Java Implementation — Reverse a Linked List

```
public static ListNode reverseList(ListNode head) {
    // Time: O(n)   Space: O(1) iterative
    ListNode prev = null;
    ListNode curr = head;
    while (curr != null) {
        ListNode nextNode = curr.next; // save the next node before overwriting
        curr.next = prev;              // reverse the pointer
        prev = curr;                   // move prev forward
        curr = nextNode;               // move curr forward
    }
    return prev;                      // prev is the new head
}
```

7.4 Step-by-Step Visual Trace

```
Original:  1 -> 2 -> 3 -> null

      prev=null curr=1
save next(2); 1.next=null; prev=1; curr=2
      null <- 1      2 -> 3 -> null

      prev=1 curr=2
save next(3); 2.next=1; prev=2; curr=3
      null <- 1 <- 2      3 -> null

      prev=2 curr=3
save next(null); 3.next=2; prev=3; curr=null
      null <- 1 <- 2 <- 3

Loop ends. Return prev = node 3 (new head).
Result: 3 -> 2 -> 1 -> null
```

7.5 Java Implementation — Detect Cycle (Floyd's Tortoise and Hare)

```
public static boolean hasCycle(ListNode head) {
    // Time: O(n)   Space: O(1)
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

```
}
```

7.6 Java Implementation — Find Middle of Linked List

```
public static ListNode findMiddle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;    // when fast reaches the end, slow is at the middle
}
```

7.7 Java Implementation — Merge Two Sorted Lists (uses dummy head)

```
public static ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(-1);
    ListNode tail = dummy;

    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) {
            tail.next = l1;
            l1 = l1.next;
        } else {
            tail.next = l2;
            l2 = l2.next;
        }
        tail = tail.next;
    }
    tail.next = (l1 != null) ? l1 : l2;    // attach leftovers
    return dummy.next;                  // skip the dummy node itself
}
```

7.8 Java Implementation — Remove Nth Node From End (one pass)

```
public static ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0, head);
    ListNode fast = dummy, slow = dummy;

    for (int i = 0; i < n; i++) fast = fast.next;    // move fast n steps ahead

    while (fast.next != null) {                      // move both until fast hits last node
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next;                      // skip the target node
    return dummy.next;
}
```

7.9 Complexity

Time: $O(n)$ for almost all single-pass linked list operations.

Space: $O(1)$ iterative, $O(n)$ if solved recursively.

7.10 Practice Problems (Easy → Hard)

Easy: Reverse Linked List · Merge Two Sorted Lists · Linked List Cycle · Middle of the Linked List

Medium: Remove Nth Node From End · Add Two Numbers · Reorder List · Odd Even Linked List · Copy List with Random Pointer

Hard: Merge K Sorted Lists · Reverse Nodes in k-Group · LRU Cache

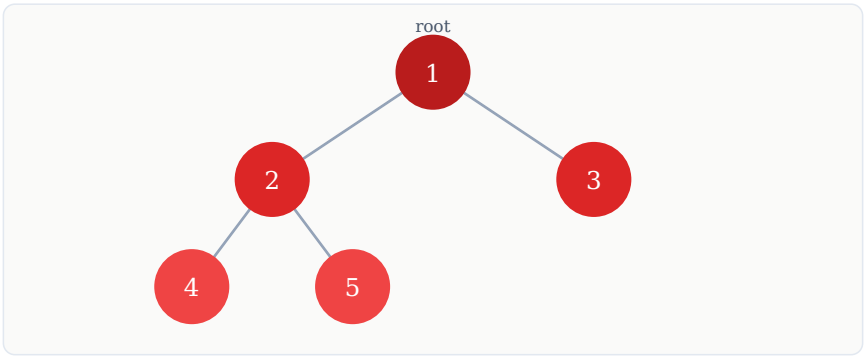
8. Binary Trees & Traversals

8.1 What it is

A tree where each node has at most two children: `left` and `right`. Almost every tree problem is a variation of a traversal combined with some computation at each node.

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val; this.left = left; this.right = right;
    }
}
```

8.2 The Four Traversal Orders



```
Preorder (Root, Left, Right): 1 -> 2 -> 4 -> 5 -> 3
Inorder (Left, Root, Right): 4 -> 2 -> 5 -> 1 -> 3    <- sorted order for a BST!
Postorder (Left, Right, Root): 4 -> 5 -> 2 -> 3 -> 1
Level-order (BFS, by depth): [1] -> [2, 3] -> [4, 5]
```

Order	Rule	Visit sequence	Typical use
Preorder	Root → Left → Right	1, 2, 4, 5, 3	Copy/serialize a tree
Inorder	Left → Root → Right	4, 2, 5, 1, 3	Get sorted values from a BST
Postorder	Left → Right → Root	4, 5, 2, 3, 1	Delete/free a tree, evaluate expression trees
Level-order	Breadth-first, layer by layer	1 2, 3 4, 5	Shortest path, printing by level

8.3 Java Implementation — Recursive Traversals

```
public static void preorder(TreeNode root, List<Integer> result) {
    if (root == null) return;
    result.add(root.val);
    preorder(root.left, result);
    preorder(root.right, result);
}

public static void inorder(TreeNode root, List<Integer> result) {
    if (root == null) return;
    inorder(root.left, result);
    result.add(root.val);
    inorder(root.right, result);
}

public static void postorder(TreeNode root, List<Integer> result) {
    if (root == null) return;
    postorder(root.left, result);
    postorder(root.right, result);
    result.add(root.val);
}
```

8.4 Java Implementation — Iterative Inorder (explicit stack)

```
public static List<Integer> inorderIterative(TreeNode root) {
    // Time: O(n)    Space: O(h)
    List<Integer> result = new ArrayList<>();
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {           // go all the way left, pushing along the way
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();              // backtrack to the last unvisited node
        result.add(curr.val);
        curr = curr.right;              // explore the right subtree
    }
    return result;
}
```

8.5 Java Implementation — Level Order Traversal (BFS with a Queue)

```
public static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> levelValues = new ArrayList<>();
        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            levelValues.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(levelValues);
    }
    return result;
}
```

8.6 Step-by-Step Visual Trace (Level Order)

Tree:

```
      1
     / \
    2   3
   / \
  4   5
```

```
queue=[1]           result=[]
pop 1, levelValues=[1], push children 2,3   queue=[2,3]
level 1 done -> result=[[1]]

pop 2, levelValues=[2], push 4,5
pop 3, levelValues=[2,3]           queue=[4,5]
level 2 done -> result=[[1],[2,3]]

pop 4, levelValues=[4]
pop 5, levelValues=[4,5]           queue=[]
level 3 done -> result=[[1],[2,3],[4,5]]
```

8.7 Java Implementation — Maximum Depth of Binary Tree

```
public static int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

8.8 Java Implementation — Check if Tree is Balanced

```
public static boolean isBalanced(TreeNode root) {  
    return height(root) != -1;  
}  
  
private static int height(TreeNode node) {  
    if (node == null) return 0;  
    int leftH = height(node.left);  
    if (leftH == -1) return -1;  
    int rightH = height(node.right);  
    if (rightH == -1) return -1;  
    if (Math.abs(leftH - rightH) > 1) return -1;  
    return 1 + Math.max(leftH, rightH);  
}
```

8.9 Java Implementation — Lowest Common Ancestor (Binary Tree)

```
public static TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {  
    if (root == null || root == p || root == q) return root;  
  
    TreeNode left = lowestCommonAncestor(root.left, p, q);  
    TreeNode right = lowestCommonAncestor(root.right, p, q);  
  
    if (left != null && right != null) return root;    // found in different subtrees  
    return left != null ? left : right;  
}
```

8.10 Complexity

Time: $O(n)$ — every node visited once for a full traversal.

Space: $O(h)$ for recursion/explicit stack, where h = tree height. Level order uses $O(w)$, w = max width.

8.11 Practice Problems (Easy → Hard)

Easy: Maximum Depth of Binary Tree · Invert Binary Tree · Same Tree · Symmetric Tree

Medium: Binary Tree Level Order Traversal · Validate Binary Search Tree · Lowest Common Ancestor · Diameter of Binary Tree · Construct Tree from Preorder+Inorder

Hard: Binary Tree Maximum Path Sum · Serialize and Deserialize Binary Tree · Vertical Order Traversal

9. Binary Search Trees (BST)

9.1 What it is

A BST maintains: for every node, all values in the left subtree are smaller, all values in the right subtree are larger. This gives $O(\log n)$ search/insert/delete on a balanced tree.

Signal words: "binary search tree", "kth smallest", "validate BST", "closest value", "range sum in BST".

9.2 Java Implementation — Search in a BST

```
public static TreeNode searchBST(TreeNode root, int target) {
    // Time: O(h)    Space: O(1) iterative
    TreeNode curr = root;
    while (curr != null) {
        if (curr.val == target) return curr;
        curr = target < curr.val ? curr.left : curr.right;
    }
    return null;
}
```

9.3 Java Implementation — Insert into a BST

```
public static TreeNode insertIntoBST(TreeNode root, int val) {
    if (root == null) return new TreeNode(val);
    if (val < root.val) {
        root.left = insertIntoBST(root.left, val);
    } else {
        root.right = insertIntoBST(root.right, val);
    }
    return root;
}
```

9.4 Step-by-Step Visual Trace — Insert 5 into a BST

```

                Start:      8
                                / \
                               3  10
                                / \
                               1  6
                                /
                               5

                Now at 3:    3
                                / \
                               1  6
                                /
                               5

                Now at 6:    6
                                /
                               5

                Insert 5: 5 < 8 -> go left
                                / \
                               3  10
                                / \
                               1  6
                                /
                               5

                Insert 5: 5 > 3 -> go right
                                / \
                               1  6
                                /
                               5

                Insert 5: 5 < 6 -> go left, left is null -> place here!
                                /
                               5

                Final tree:
                                8
                                / \
                               3  10
                                / \
                               1  6
                                /
                               5
```

9.5 Java Implementation — Validate BST (bounds, not just parent comparison)

```
public static boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

private static boolean validate(TreeNode node, long low, long high) {
    if (node == null) return true;
```

```

    if (node.val <= low || node.val >= high) return false;
    return validate(node.left, low, node.val) && validate(node.right, node.val, high);
}

```

Common mistake: Comparing a node only to its immediate parent is **wrong**. A node deep in the left subtree must be smaller than ALL its ancestors above it. Always pass down a valid (low, high) range.

9.6 Java Implementation — Kth Smallest Element in BST

```

public static int kthSmallest(TreeNode root, int k) {
    // Time: O(h + k)    Space: O(h)
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;
    int count = 0;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();
        count++;
        if (count == k) return curr.val;
        curr = curr.right;
    }
    return -1;    // k was out of range
}

```

9.7 Complexity

Time: $O(\log n)$ average (balanced tree), $O(n)$ worst case (skewed tree).

Space: $O(h)$ for recursion.

Tip: Java's `TreeMap`/`TreeSet` are self-balancing (Red-Black trees) and guarantee $O(\log n)$ always — mention them when discussing production-quality alternatives.

9.8 Practice Problems (Easy → Hard)

Easy: Search in a BST · Insert into a BST · Minimum Absolute Difference in BST

Medium: Validate BST · Kth Smallest Element in BST · Lowest Common Ancestor of a BST · Delete Node in a BST · Convert Sorted Array to BST

Hard: Recover Binary Search Tree · Balance a BST · Count of Smaller Numbers After Self

10. Heaps & Top-K Elements

10.1 What a heap is

A binary heap is a complete binary tree where every parent is smaller (min-heap) or larger (max-heap) than its children. Java's `PriorityQueue` is a min-heap by default.

Signal words: "kth largest/smallest", "top K frequent", "K closest points", "merge K sorted lists", "median of a stream", "task scheduler".

10.2 Java's PriorityQueue

```
import java.util.PriorityQueue;
import java.util.Collections;

PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // min-heap (default)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // max-heap

minHeap.offer(5); // O(log n)
minHeap.peek(); // O(1) -- smallest element
minHeap.poll(); // O(log n) -- remove & return smallest
```

10.3 Java Implementation — Kth Largest Element (min-heap of size K)

```
public static int findKthLargest(int[] nums, int k) {
    // Time: O(n log k) Space: O(k)
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int num : nums) {
        heap.offer(num);
        if (heap.size() > k) heap.poll(); // remove smallest, keep top k largest
    }
    return heap.peek(); // smallest of the k largest = kth largest
}
```

10.4 Step-by-Step Visual Trace

nums = [3, 2, 1, 5, 6, 4], k = 2 (find 2nd largest)

	push 3	heap=[3]	
	push 2	heap=[2,3]	
push 1	heap=[1,2,3]	size=3 > k=2 -> pop smallest(1)	heap=[2,3]
push 5	heap=[2,3,5]	size=3 > k=2 -> pop smallest(2)	heap=[3,5]
push 6	heap=[3,5,6]	size=3 > k=2 -> pop smallest(3)	heap=[5,6]
push 4	heap=[4,5,6]	size=3 > k=2 -> pop smallest(4)	heap=[5,6]

Final heap = [5, 6] -> peek() = 5 -> the 2nd largest element = 5

10.5 Java Implementation — Top K Frequent Elements

```
public static int[] topKFrequent(int[] nums, int k) {
    // Time: O(n log k)
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : nums) freq.put(num, freq.getOrDefault(num, 0) + 1);

    PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) -> freq.get(a) - freq.get(b));
    for (int num : freq.keySet()) {
        heap.offer(num);
        if (heap.size() > k) heap.poll();
    }

    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) result[i] = heap.poll();
    return result;
}
```

10.6 Java Implementation — Merge K Sorted Lists (heap of pointers)

```
public static ListNode mergeKLists(ListNode[] lists) {
    // Time: O(N log k)    Space: O(k)
    PriorityQueue<ListNode> heap = new PriorityQueue<>((a, b) -> a.val - b.val);
    for (ListNode node : lists) {
        if (node != null) heap.offer(node);
    }

    ListNode dummy = new ListNode(-1);
    ListNode tail = dummy;

    while (!heap.isEmpty()) {
        ListNode node = heap.poll();
        tail.next = node;
        tail = tail.next;
        if (node.next != null) heap.offer(node.next);
    }
    return dummy.next;
}
```

10.7 Java Implementation — Median of a Data Stream (two heaps)

```
class MedianFinder {
    private PriorityQueue<Integer> small;    // max-heap, holds the smaller half
    private PriorityQueue<Integer> large;    // min-heap, holds the larger half

    public MedianFinder() {
        small = new PriorityQueue<>(Collections.reverseOrder());
        large = new PriorityQueue<>();
    }

    public void addNum(int num) {
        small.offer(num);
        large.offer(small.poll());
        if (large.size() > small.size()) {
            small.offer(large.poll());
        }
    }

    public double findMedian() {
        if (small.size() > large.size()) return small.peek();
        return (small.peek() + large.peek()) / 2.0;
    }
}
```

10.8 Complexity

Time: $O(\log n)$ for offer/poll, $O(n)$ to build a heap from a collection, $O(n \log k)$ for typical top-K problems.
Space: $O(n)$ for the heap, or $O(k)$ for a bounded-size heap.

10.9 Practice Problems (Easy → Hard)

Easy: Kth Largest Element in a Stream · Last Stone Weight

Medium: Kth Largest Element in an Array · Top K Frequent Elements · K Closest Points to Origin · Task Scheduler · Reorganize String

Hard: Merge K Sorted Lists · Find Median from Data Stream · Smallest Range Covering Elements from K Lists

11. Graphs: BFS & DFS

11.1 What it is

A graph is a set of nodes (vertices) connected by edges. Trees, grids, and state spaces can all be modeled as graphs.

Algorithm	Data structure used	Best for
BFS (Breadth-First Search)	Queue	Shortest path in unweighted graph, level-by-level processing
DFS (Depth-First Search)	Stack / Recursion	Exploring all paths, connectivity, cycle detection, topological sort

Signal words: "shortest path" (unweighted → BFS), "connected components", "all paths", "cycle detection", "course schedule / dependency order" (topological sort).

11.2 Representing a Graph in Java

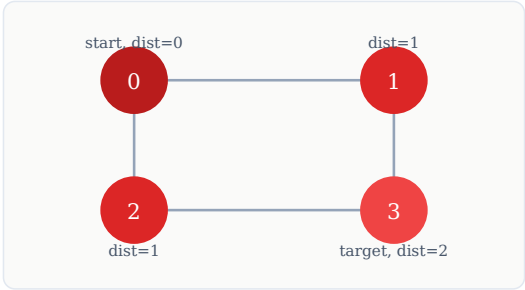
```
Map<Integer, List<Integer>> graph = new HashMap<>();
int[][] edges = {{0,1}, {0,2}, {1,3}, {2,3}};
for (int[] edge : edges) {
    int u = edge[0], v = edge[1];
    graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
    graph.computeIfAbsent(v, k -> new ArrayList<>()).add(u);    // remove for DIRECTED graph
}
```

11.3 Java Implementation — BFS (Shortest Path, Unweighted)

```
public static int bfsShortestPath(Map<Integer, List<Integer>> graph, int start, int target) {
    // Time: O(V + E)    Space: O(V)
    Set<Integer> visited = new HashSet<>();
    Queue<int[]> queue = new LinkedList<>();    // {node, distance}
    visited.add(start);
    queue.offer(new int[] {start, 0});

    while (!queue.isEmpty()) {
        int[] curr = queue.poll();
        int node = curr[0], dist = curr[1];
        if (node == target) return dist;
        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.offer(new int[] {neighbor, dist + 1});
            }
        }
    }
    return -1;    // target not reachable
}
```

11.4 Step-by-Step Visual Trace (BFS)



Graph:

0 --- 1
| |
2 --- 3

Find shortest path 0 -> 3

```

        queue=[(0,0)] visited={0}
        pop (0,0): neighbors 1,2 -> add both, dist=1
        queue=[(1,1),(2,1)] visited={0,1,2}

        pop (1,1): neighbors 0(visited),3(new) -> add 3, dist=2
        queue=[(2,1),(3,2)] visited={0,1,2,3}

        pop (2,1): neighbors 0(visited),3(visited) -> nothing new
        queue=[(3,2)]

        pop (3,2): node == target! return dist = 2

```

11.5 Java Implementation — DFS (Recursive)

```

public static Set<Integer> dfsRecursive(Map<Integer, List<Integer>> graph, int node, Set<Integer> visited) {
    visited.add(node);
    System.out.println(node);           // process node

    for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
        if (!visited.contains(neighbor)) {
            dfsRecursive(graph, neighbor, visited);
        }
    }
    return visited;
}

```

11.6 Java Implementation — DFS (Iterative, explicit stack)

```

public static Set<Integer> dfsIterative(Map<Integer, List<Integer>> graph, int start) {
    Set<Integer> visited = new HashSet<>();
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(start);

    while (!stack.isEmpty()) {
        int node = stack.pop();
        if (!visited.contains(node)) {
            visited.add(node);
            for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
                if (!visited.contains(neighbor)) stack.push(neighbor);
            }
        }
    }
    return visited;
}

```

11.7 Java Implementation — Detect Cycle in Directed Graph

```

public static boolean hasCycle(Map<Integer, List<Integer>> graph, int numNodes) {
    // 0 = unvisited, 1 = visiting, 2 = done. Time: O(V+E)
    int[] state = new int[numNodes];

    for (int node = 0; node < numNodes; node++) {
        if (state[node] == 0 && dfsHasCycle(graph, node, state)) return true;
    }
    return false;
}

private static boolean dfsHasCycle(Map<Integer, List<Integer>> graph, int node, int[] state) {
    state[node] = 1;
    for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
        if (state[neighbor] == 1) return true;           // back edge -> cycle!
        if (state[neighbor] == 0 && dfsHasCycle(graph, neighbor, state)) return true;
    }
    state[node] = 2;
    return false;
}

```

11.8 Java Implementation — Topological Sort (Kahn's BFS algorithm)

```

public static List<Integer> topologicalSort(int numNodes, int[][] edges) {
    // Time: O(V + E)
    Map<Integer, List<Integer>> graph = new HashMap<>();
    int[] indegree = new int[numNodes];

    for (int[] edge : edges) {
        // edge u -> v means "u before v"
        int u = edge[0], v = edge[1];
        graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
        indegree[v]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numNodes; i++) if (indegree[i] == 0) queue.offer(i);

    List<Integer> order = new ArrayList<>();
    while (!queue.isEmpty()) {
        int node = queue.poll();
        order.add(node);
        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (--indegree[neighbor] == 0) queue.offer(neighbor);
        }
    }
    return order.size() == numNodes ? order : new ArrayList<>(); // empty if a cycle exists
}

```

11.9 Complexity

Time: $O(V + E)$ for both BFS and DFS.

Space: $O(V)$ for the visited set / queue / recursion stack.

11.10 Practice Problems (Easy → Hard)

Easy: Find if Path Exists in Graph · Flood Fill · Number of Provinces

Medium: Number of Islands · Clone Graph · Course Schedule (I & II) · Rotting Oranges (multi-source BFS) · Pacific Atlantic Water Flow

Hard: Word Ladder · Alien Dictionary · Critical Connections in a Network · Minimum Height Trees

12. Backtracking

12.1 What it is

Backtracking explores all possible choices via recursion, and **undoes** a choice when it leads to a dead end — DFS over a "decision tree" of choices, with pruning to skip invalid branches early.

Signal words: "all permutations", "all subsets/combinations", "all valid arrangements", "N-Queens", "Sudoku solver", "generate parentheses", "word search".

12.2 The Universal Backtracking Template

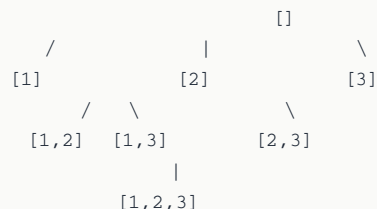
```
void backtrack(List<Integer> path, List<Integer> choices) {
    if (isSolution(path)) {
        result.add(new ArrayList<>(path));    // record a valid full solution (COPY it!)
        return;
    }
    for (int choice : choices) {
        if (!isValid(choice, path)) continue;    // prune invalid branches early
        path.add(choice);                        // MAKE the choice
        backtrack(path, nextChoices);           // EXPLORE further
        path.remove(path.size() - 1);           // UNDO the choice (backtrack!)
    }
}
```

12.3 Java Implementation — Subsets (Power Set)

```
public static List<List<Integer>> subsets(int[] nums) {
    // Time: O(2^n * n)    Space: O(n) recursion depth
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private static void backtrack(int[] nums, int start, List<Integer> path, List<List<Integer>> result) {
    result.add(new ArrayList<>(path));    // every path is a valid subset
    for (int i = start; i < nums.length; i++) {
        path.add(nums[i]);                // choose
        backtrack(nums, i + 1, path, result);    // explore
        path.remove(path.size() - 1);          // un-choose
    }
}
```

12.4 Step-by-Step Visual Trace — Subsets of [1,2,3]



Every node in this tree (including the root []) is a valid subset.

Order visited (DFS/preorder):

[] -> [1] -> [1,2] -> [1,2,3] -> backtrack -> [1,3] -> backtrack
-> [2] -> [2,3] -> backtrack -> [3] -> backtrack -> done

Total subsets = $2^3 = 8$

12.5 Java Implementation — Permutations

```
public static List<List<Integer>> permute(int[] nums) {
```

```

// Time: O(n! * n)
List<List<Integer>> result = new ArrayList<>();
backtrackPermute(nums, new ArrayList<>(), new boolean[nums.length], result);
return result;
}

private static void backtrackPermute(int[] nums, List<Integer> path, boolean[] used, List<List<Integer>> result) {
    if (path.size() == nums.length) {
        result.add(new ArrayList<>(path));
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        used[i] = true;
        path.add(nums[i]);
        backtrackPermute(nums, path, used, result);
        path.remove(path.size() - 1);
        used[i] = false;
    }
}

```

12.6 Java Implementation — Combination Sum

```

public static List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(candidates);
    backtrackSum(candidates, target, 0, new ArrayList<>(), result);
    return result;
}

private static void backtrackSum(int[] candidates, int remaining, int start,
                                List<Integer> path, List<List<Integer>> result) {
    if (remaining == 0) {
        result.add(new ArrayList<>(path));
        return;
    }
    for (int i = start; i < candidates.length; i++) {
        if (candidates[i] > remaining) break;           // prune: sorted, no point continuing
        path.add(candidates[i]);
        backtrackSum(candidates, remaining - candidates[i], i, path, result); // i, reuse allowed
        path.remove(path.size() - 1);
    }
}

```

12.7 Java Implementation — N-Queens

```

public static List<List<String>> solveNQueens(int n) {
    List<List<String>> result = new ArrayList<>();
    Set<Integer> cols = new HashSet<>(), diag1 = new HashSet<>(), diag2 = new HashSet<>();
    char[][] board = new char[n][n];
    for (char[] row : board) Arrays.fill(row, '.');

    backtrackQueens(0, n, cols, diag1, diag2, board, result);
    return result;
}

private static void backtrackQueens(int row, int n, Set<Integer> cols, Set<Integer> diag1,
                                    Set<Integer> diag2, char[][] board, List<List<String>> result) {
    if (row == n) {
        List<String> solution = new ArrayList<>();
        for (char[] r : board) solution.add(new String(r));
        result.add(solution);
        return;
    }
    for (int col = 0; col < n; col++) {
        if (cols.contains(col) || diag1.contains(row - col) || diag2.contains(row + col)) continue;
        cols.add(col); diag1.add(row - col); diag2.add(row + col);
        board[row][col] = 'Q';

        backtrackQueens(row + 1, n, cols, diag1, diag2, board, result);

        cols.remove(col); diag1.remove(row - col); diag2.remove(row + col);
        board[row][col] = '.';
    }
}

```

12.8 Complexity

Time: Exponential in general — $O(2^n)$ for subsets, $O(n!)$ for permutations. Pruning makes backtracking practical.
Space: $O(n)$ for the recursion stack and current path, plus output storage.

12.9 Practice Problems (Easy → Hard)

Easy: Binary Watch · Letter Case Permutation

Medium: Subsets · Permutations · Combination Sum · Generate Parentheses · Word Search · Palindrome Partitioning

Hard: N-Queens · Sudoku Solver · Word Search II (with Trie) · Expression Add Operators

13. Greedy Algorithms

13.1 What it is

A greedy algorithm makes the **locally optimal choice** at every step, hoping it leads to a globally optimal solution. Works only when the problem has the **greedy-choice property**.

Signal words: "maximum/minimum number of...", "non-overlapping", "minimum coins", "assign/schedule to maximize", "jump game".

13.2 Java Implementation — Non-overlapping Intervals

```
public static int eraseOverlapIntervals(int[][] intervals) {
    // Time: O(n log n)    Space: O(1) extra
    if (intervals.length == 0) return 0;

    Arrays.sort(intervals, (a, b) -> a[1] - b[1]);    // sort by END time
    int countRemoved = 0;
    int lastEnd = intervals[0][1];

    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < lastEnd) {
            countRemoved++;                          // overlaps -> remove this one
        } else {
            lastEnd = intervals[i][1];                 // no overlap, keep it
        }
    }
    return countRemoved;
}
```

13.3 Step-by-Step Visual Trace

Intervals = `[[1,2], [2,3], [3,4], [1,3]]`

```
Sorted by end time: [1,2] [2,3] [1,3] [3,4]

lastEnd = 2 (from [1,2])
[2,3]: start=2 >= lastEnd=2 -> no overlap, keep, lastEnd=3
[1,3]: start=1 < lastEnd=3 -> OVERLAP! remove this one. count=1
[3,4]: start=3 >= lastEnd=3 -> no overlap, keep, lastEnd=4

Answer: 1 interval removed ([1,3])
```

13.4 Java Implementation — Jump Game

```
public static boolean canJump(int[] nums) {
    // Time: O(n)
    int farthest = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > farthest) return false;
        farthest = Math.max(farthest, i + nums[i]);
    }
    return true;
}
```

13.5 Java Implementation — Gas Station

```
public static int canCompleteCircuit(int[] gas, int[] cost) {
    // Time: O(n)
    int totalTank = 0, currTank = 0, start = 0;

    for (int i = 0; i < gas.length; i++) {
        int diff = gas[i] - cost[i];
        totalTank += diff;
    }
```

```

currTank += diff;
if (currTank < 0) {
    start = i + 1;    // this segment failed, try starting fresh after it
    currTank = 0;
}
}
return totalTank >= 0 ? start : -1;
}

```

13.6 Java Implementation — Meeting Rooms II (Minimum Rooms Needed)

```

public static int minMeetingRooms(int[][] intervals) {
    // Time: O(n log n)
    if (intervals.length == 0) return 0;

    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    PriorityQueue<Integer> heap = new PriorityQueue<>();    // stores end times

    for (int[] interval : intervals) {
        if (!heap.isEmpty() && heap.peek() <= interval[0]) {
            heap.poll();    // reuse the room that already freed up
        }
        heap.offer(interval[1]);
    }
    return heap.size();
}

```

13.7 Greedy vs Dynamic Programming

Greedy	Dynamic Programming
One irrevocable choice per step	Considers all choices, remembers overlapping subproblems
Usually $O(n \log n)$ or $O(n)$	Usually $O(n^2)$ or higher
Only works if greedy-choice property holds	Always works if optimal substructure holds
Example: Interval Scheduling	Example: 0/1 Knapsack (greedy fails here!)

13.8 Complexity

Time: Usually $O(n \log n)$ due to sorting, then $O(n)$ for a single greedy pass.
Space: $O(1)$ to $O(n)$ depending on extra structures needed (e.g. a heap).

13.9 Practice Problems (Easy → Hard)

Easy: Assign Cookies · Lemonade Change · Best Time to Buy and Sell Stock II
Medium: Jump Game · Gas Station · Non-overlapping Intervals · Partition Labels · Meeting Rooms II
Hard: Candy · Minimum Number of Arrows to Burst Balloons · Job Sequencing with Deadlines

14. Dynamic Programming (DP)

14.1 What it is

DP solves a problem by breaking it into overlapping subproblems, solving each once, and reusing stored results. Requires:

- **Optimal substructure:** the optimal full solution builds from optimal subproblem solutions.
- **Overlapping subproblems:** the same subproblem is solved multiple times naively.

Signal words: "maximum/minimum ways to...", "number of distinct ways", "longest/shortest subsequence", "can you partition into...", "minimum cost to reach..."

14.2 The DP Thinking Framework

Step 1 — Define the state: What does `dp[i]` represent in plain English?

Step 2 — Find the recurrence: How does `dp[i]` relate to smaller states?

Step 3 — Base case(s): What is the answer for the smallest input?

Step 4 — Order of computation: Bottom-up (table) or top-down (recursion + memo)?

Step 5 — Final answer: Which cell of the table holds it?

14.3 Classic Example — Fibonacci (all 3 approaches)

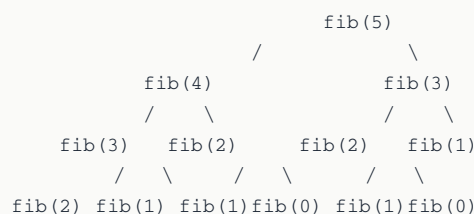
```
// 1) Naive recursion -- O(2^n) -- exponential, recomputes same values!
public static int fibNaive(int n) {
    if (n <= 1) return n;
    return fibNaive(n - 1) + fibNaive(n - 2);
}

// 2) Top-down memoization -- O(n) time, O(n) space
public static int fibMemo(int n, Map<Integer, Integer> cache) {
    if (n <= 1) return n;
    if (cache.containsKey(n)) return cache.get(n);
    int result = fibMemo(n - 1, cache) + fibMemo(n - 2, cache);
    cache.put(n, result);
    return result;
}

// 3) Bottom-up tabulation -- O(n) time, O(1) space (rolling variables)
public static int fibBottomUp(int n) {
    if (n <= 1) return n;
    int prev2 = 0, prev1 = 1;
    for (int i = 2; i <= n; i++) {
        int curr = prev2 + prev1;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

14.4 Visual: Why Memoization Helps (Fibonacci call tree)

Naive recursion tree for `fib(5)` -- notice `fib(3)`, `fib(2)` computed MULTIPLE times:



```

      / \
    fib(1)fib(0)

```

fib(2) is computed 3 times, fib(3) is computed 2 times -- wasted work!
Memoization stores each result the FIRST time and reuses it after that.

14.5 1D DP — House Robber

```

public static int rob(int[] nums) {
    // dp[i] = max money robbed from houses[0..i]
    // Recurrence: dp[i] = max(dp[i-1], dp[i-2] + nums[i])
    // Time: O(n)    Space: O(1) after rolling optimization
    int prev2 = 0, prev1 = 0;
    for (int num : nums) {
        int curr = Math.max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

```

```

        nums = [2, 7, 9, 3, 1]
        dp:    [2, 7, 11, 11, 12]

```

```

        dp[0]=2
        dp[1]=max(2,7)=7
        dp[2]=max(7, 2+9)=11
        dp[3]=max(11, 7+3)=11
        dp[4]=max(11, 11+1)=12    <-- answer

```

14.6 1D DP — Longest Increasing Subsequence (LIS)

```

public static int lengthOfLIS(int[] nums) {
    // dp[i] = length of LIS ENDING at index i
    // Time: O(n^2)    Space: O(n)
    if (nums.length == 0) return 0;
    int[] dp = new int[nums.length];
    Arrays.fill(dp, 1);
    int maxLen = 1;

    for (int i = 0; i < nums.length; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
        }
        maxLen = Math.max(maxLen, dp[i]);
    }
    return maxLen;
}

```

14.7 2D DP — Longest Common Subsequence (LCS)

```

public static int longestCommonSubsequence(String text1, String text2) {
    // dp[i][j] = LCS length of text1[0..i-1] and text2[0..j-1]
    // Time: O(m*n)    Space: O(m*n)
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i-1) == text2.charAt(j-1)) {
                dp[i][j] = 1 + dp[i-1][j-1];
            } else {
                dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }
    return dp[m][n];
}

```

14.8 Step-by-Step DP Table Trace — LCS("abcde", "ace")

		a	c	e
a	0	1	1	1
b	0	1	1	1
c	0	1	2	2
d	0	1	2	2
e	0	1	2	3

Reading the table: whenever row-char == col-char, take diagonal+1.
Otherwise, take the max of the cell above or to the left.

14.9 2D DP — 0/1 Knapsack

```
public static int knapsack(int[] weights, int[] values, int capacity) {
    // dp[i][w] = max value using first i items with capacity w
    // Time: O(n * capacity)    Space: O(n * capacity)
    int n = weights.length;
    int[][] dp = new int[n + 1][capacity + 1];

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i][w] = dp[i-1][w]; // don't take item i-1
            if (weights[i-1] <= w) {
                dp[i][w] = Math.max(dp[i][w], dp[i-1][w - weights[i-1]] + values[i-1]);
            }
        }
    }
    return dp[n][capacity];
}
```

14.10 2D DP — Edit Distance

```
public static int minDistance(String word1, String word2) {
    // dp[i][j] = min operations to convert word1[0..i-1] to word2[0..j-1]
    // Time: O(m*n)
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 0; i <= m; i++) dp[i][0] = i; // delete all i characters
    for (int j = 0; j <= n; j++) dp[0][j] = j; // insert all j characters

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i-1) == word2.charAt(j-1)) {
                dp[i][j] = dp[i-1][j-1];
            } else {
                dp[i][j] = 1 + Math.min(dp[i-1][j], Math.min(dp[i][j-1], dp[i-1][j-1]));
            }
        }
    }
    return dp[m][n];
}
```

14.11 DP on Grids — Unique Paths

```
public static int uniquePaths(int m, int n) {
    // dp[r][c] = ways to reach (r,c) from (0,0), moving only right/down
    // Time: O(m*n)    Space: O(n) with row-compression
    int[] dp = new int[n];
    Arrays.fill(dp, 1); // first row is always all 1s

    for (int r = 1; r < m; r++) {
        for (int c = 1; c < n; c++) {
            dp[c] += dp[c - 1]; // dp[c] currently holds dp[r-1][c]
        }
    }
}
```

```
    return dp[n - 1];  
}
```

14.12 DP Category Cheat Sheet

Category	Classic Problem	State
1D linear	House Robber, Climbing Stairs	dp[i] depends on dp[i-1], dp[i-2]
Subsequence	LIS, LCS	dp[i] or dp[i][j] tracks best ending/matching
Knapsack	Partition Equal Subset Sum, Coin Change	dp[i][capacity]
String matching/edit	Edit Distance, Wildcard Matching	dp[i][j] over two strings
Grid path	Unique Paths, Minimum Path Sum	dp[r][c]
Interval DP	Burst Balloons, Matrix Chain Multiplication	dp[i][j] over a range
State machine DP	Best Time to Buy/Sell Stock with Cooldown	dp[i][state]

14.13 Complexity

Time: Typically $O(n)$, $O(n^2)$, or $O(n \times \text{capacity})$ depending on states and transitions.
Space: $O(\text{number of states})$; often reducible via "rolling array" optimization.

14.14 Practice Problems (Easy → Hard)

Easy: Climbing Stairs · House Robber · Maximum Subarray (Kadane's) · Min Cost Climbing Stairs
Medium: Coin Change · Longest Increasing Subsequence · Longest Common Subsequence · Unique Paths · Word Break · Partition Equal Subset Sum
Hard: Edit Distance · Regular Expression Matching · Burst Balloons · Best Time to Buy/Sell Stock with Cooldown · Interleaving String · Palindrome Partitioning II

15. Overlapping Intervals

15.1 What it is

Problems with a list of `[start, end]` ranges asking you to merge, insert, count overlaps, or find free/busy time. The universal first step is almost always: **sort by start time**.

Signal words: "merge intervals", "insert interval", "overlapping meetings", "free time", "employee schedules".

15.2 How to Tell if Two Intervals Overlap

```
Interval A: [aStart ----- aEnd]
Interval B:      [bStart ----- bEnd]

A and B overlap if:  aStart <= bEnd  AND  bStart <= aEnd
```

15.3 Java Implementation — Merge Intervals

```
public static int[][] merge(int[][] intervals) {
    // Time: O(n log n)    Space: O(n)
    if (intervals.length == 0) return new int[0][];

    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> merged = new ArrayList<>();
    merged.add(intervals[0]);

    for (int i = 1; i < intervals.length; i++) {
        int[] last = merged.get(merged.size() - 1);
        if (intervals[i][0] <= last[1]) {
            last[1] = Math.max(last[1], intervals[i][1]); // extend it
        } else {
            merged.add(intervals[i]); // start a new interval
        }
    }
    return merged.toArray(new int[0][]);
}
```

15.4 Step-by-Step Visual Trace

Intervals = `[[1,3],[2,6],[8,10],[15,18]]`

```
Sorted (already sorted by start): [1,3] [2,6] [8,10] [15,18]

merged = [[1,3]]
[2,6]: 2 <= 3 (last_end) -> overlap -> extend: merged=[[1,6]]
[8,10]: 8 <= 6? NO -> no overlap -> append: merged=[[1,6],[8,10]]
[15,18]: 15 <= 10? NO -> no overlap -> append: merged=[[1,6],[8,10],[15,18]]

Final: [[1,6],[8,10],[15,18]]
```

15.5 Java Implementation — Insert Interval

```
public static int[][] insert(int[][] intervals, int[] newInterval) {
    // Time: O(n)
    List<int[]> result = new ArrayList<>();
    int i = 0, n = intervals.length;

    while (i < n && intervals[i][1] < newInterval[0]) { // before newInterval
        result.add(intervals[i++]);
    }
    while (i < n && intervals[i][0] <= newInterval[1]) { // overlapping
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
    }
    result.add(newInterval);
    while (i < n) {
        result.add(intervals[i++]);
    }
    return result.toArray(new int[0][]);
}
```

```

        i++;
    }
    result.add(newInterval);
    while (i < n) result.add(intervals[i++]); // after newInterval

    return result.toArray(new int[0] []);
}

```

15.6 Java Implementation — Meeting Rooms I (can attend all meetings?)

```

public static boolean canAttendMeetings(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < intervals[i-1][1]) return false;
    }
    return true;
}

```

15.7 Complexity

Time: $O(n \log n)$ when sorting is required, $O(n)$ if already sorted.

Space: $O(n)$ for the output list.

15.8 Practice Problems (Easy → Hard)

Easy: Meeting Rooms I · Summary Ranges

Medium: Merge Intervals · Insert Interval · Non-overlapping Intervals · Meeting Rooms II

Hard: Employee Free Time · Minimum Number of Arrows to Burst Balloons · Data Stream as Disjoint Intervals

16. Binary Search (Including "Binary Search on Answer")

16.1 Classic Binary Search

```
public static int binarySearch(int[] nums, int target) {
    // Time: O(log n)    Space: O(1)
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;    // avoids overflow
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

16.2 Step-by-Step Visual Trace

nums = [1, 3, 5, 7, 9, 11], target = 9

```

    [1, 3, 5, 7, 9, 11]    left=0 right=5 mid=2 (val=5)    5<9 -> left=3
                        L  M  R
[. . . 7, 9, 11]    left=3 right=5 mid=4 (val=9)    9==9 -> FOUND at index 4
                        L  M  R
```

16.3 The "Binary Search on Answer" Pattern

When a problem asks for the **minimum/maximum value satisfying a monotonic condition**, you can binary search over the *answer space* itself, not the array.

Signal words: "minimize the maximum...", "maximize the minimum...", "smallest capacity/speed/days such that..."

16.4 Pseudocode

```
int binarySearchOnAnswer(int low, int high, Predicate<Integer> feasible) {
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (feasible.test(mid)) {
            high = mid;    // mid works, try to do even better
        } else {
            low = mid + 1;    // mid doesn't work, need a bigger value
        }
    }
    return low;
}
```

16.5 Java Implementation — Koko Eating Bananas

```
public static int minEatingSpeed(int[] piles, int h) {
    // feasible(k) = True if eating at speed k finishes within h hours
    // Time: O(n log(max(piles)))
    int low = 1, high = 0;
    for (int pile : piles) high = Math.max(high, pile);

    while (low < high) {
        int mid = low + (high - low) / 2;
        if (hoursNeeded(piles, mid) <= h) {
            high = mid;    // speed mid works, try slower
        } else {
            low = mid + 1;    // too slow, need to eat faster
        }
    }
    return low;
}
```

```

}

private static long hoursNeeded(int[] piles, int k) {
    long hours = 0;
    for (int pile : piles) hours += Math.ceil((double) pile / k);
    return hours;
}

```

16.6 Step-by-Step Visual Trace

piles = [3,6,7,11], h = 8

```

                                low=1 high=11
mid=6:  hours = ceil(3/6)+ceil(6/6)+ceil(7/6)+ceil(11/6) = 1+1+2+2 = 6 <= 8 -> feasible, high=6
                                low=1 high=6
mid=3:  hours = 1+2+3+4 = 10 > 8 -> not feasible, low=4
                                low=4 high=6
mid=5:  hours = 1+2+2+3 = 8 <= 8 -> feasible, high=5
                                low=4 high=5
mid=4:  hours = 1+2+2+3 = 8 <= 8 -> feasible, high=4
                                low=4 high=4 -> loop ends, answer = 4

```

16.7 Java Implementation — Search in Rotated Sorted Array

```

public static int searchRotated(int[] nums, int target) {
    // Time: O(log n)
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (nums[mid] == target) return mid;

        if (nums[left] <= nums[mid]) { // left half sorted
            if (nums[left] <= target && target < nums[mid]) right = mid - 1;
            else left = mid + 1;
        } else { // right half sorted
            if (nums[mid] < target && target <= nums[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}

```

16.8 Complexity

Time: $O(\log n)$ for the search itself; $O(n \log(\text{range}))$ for binary-search-on-answer.

Space: $O(1)$.

16.9 Practice Problems (Easy → Hard)

Easy: Binary Search · Search Insert Position · First Bad Version

Medium: Search in Rotated Sorted Array · Find Peak Element · Koko Eating Bananas · Find Minimum in Rotated Sorted Array

Hard: Median of Two Sorted Arrays · Split Array Largest Sum · Capacity to Ship Packages Within D Days

17. Trie (Prefix Tree)

A tree structure specialized for strings, where each path from the root spells out a prefix. Nodes are shared between words with common prefixes.

```

Words inserted: "cat", "car", "dog"

      root
     /   \
    c     d
    |     |
    a     o
   / \   \
  t  r   g
*  *   *   (* marks end of a valid word)

"cat" and "car" share the path root -> c -> a

```

```

Words inserted: "cat", "car", "dog"

      root
     /   \
    c     d
    |     |
    a     o
   / \   \
  t  r   g
*  *   *   (* marks end of a valid word)

"cat" and "car" share the path root -> c -> a

```

Signal words: "autocomplete", "prefix matching", "implement a dictionary of words", "word search with a list of words".

Time: $O(L)$ for insert/search/prefix-check, L = length of word/prefix — independent of how many words are stored!
Space: $O(\text{total characters across all inserted words})$ worst case.

Time: $O(L)$ for insert/search/prefix-check, L = length of word/prefix — independent of how many words are stored!
Space: $O(\text{total characters across all inserted words})$ worst case.

Easy: Implement Trie (Prefix Tree) · Longest Common Prefix

Medium: Design Add and Search Words Data Structure · Replace Words · Map Sum Pairs

Hard: Word Search II · Palindrome Pairs · Stream of Characters

18. Bit Manipulation

18.1 Core Operators

Operator	Meaning	Example
&	AND	5 & 3 = 101 & 011 = 001 = 1
	OR	5 3 = 101 011 = 111 = 7
^	XOR	5 ^ 3 = 101 ^ 011 = 110 = 6
~	NOT	~5 = -6 (two's complement)
<<	Left shift	5 << 1 = 10
>>	Right shift (sign-extending)	5 >> 1 = 2
>>>	Unsigned right shift	-8 >>> 1 fills with 0, not sign bit

Signal words: "single number", "count set bits", "power of two", "without using +/- operators", "subsets via bitmask".

18.2 Handy Bit Tricks

```
n & (n - 1)    // removes the lowest set bit
n & (-n)       // isolates the lowest set bit
n ^ n == 0     // XOR-ing a number with itself cancels out to 0
n ^ 0 == n     // XOR-ing with 0 does nothing
```

18.3 Java Implementation — Single Number

```
public static int singleNumber(int[] nums) {
    // Time: O(n)    Space: O(1)
    int result = 0;
    for (int num : nums) result ^= num;
    return result;
}
```

18.4 Step-by-Step Visual Trace

nums = [4, 1, 2, 1, 2]

result = 0

0 ^ 4 = 4 (0100)

4 ^ 1 = 5 (0101)

5 ^ 2 = 7 (0111)

7 ^ 1 = 6 (0110)

6 ^ 2 = 4 (0100) <- final result = 4 (the unique number)

18.5 Java Implementation — Count Set Bits (Hamming Weight)

```
public static int hammingWeight(int n) {
    // Time: O(number of set bits)
    int count = 0;
    while (n != 0) {
        n &= (n - 1);    // removes the lowest set bit each time
        count++;
    }
    return count;
}
// Java also has a built-in: Integer.bitCount(n)
```

18.6 Java Implementation — Check Power of Two

```
public static boolean isPowerOfTwo(int n) {  
    return n > 0 && (n & (n - 1)) == 0;  
}
```

18.7 Java Implementation — Generate All Subsets Using Bitmasks

```
public static List<List<Integer>> subsetsViaBitmask(int[] nums) {  
    // Time: O(2^n * n)  
    int n = nums.length;  
    List<List<Integer>> result = new ArrayList<>();  
  
    for (int mask = 0; mask < (1 << n); mask++) {  
        List<Integer> subset = new ArrayList<>();  
        for (int i = 0; i < n; i++) {  
            if ((mask & (1 << i)) != 0) subset.add(nums[i]);  
        }  
        result.add(subset);  
    }  
    return result;  
}
```

18.8 Complexity

Time: O(1) for basic operations, O(log n) or O(number of bits) for bit-counting loops.

Space: O(1) — bit tricks use no extra memory.

18.9 Practice Problems (Easy → Hard)

Easy: Single Number · Number of 1 Bits · Power of Two · Counting Bits

Medium: Single Number II/III · Sum of Two Integers (no +/-) · Subsets via bitmask

Hard: Maximum XOR of Two Numbers in an Array · Minimum XOR Sum of Two Arrays

19. Union-Find (Disjoint Set Union)

19.1 What it is

Tracks which elements belong to the same group, supporting **find** (which group?) and **union** (merge two groups) very efficiently. With **path compression** and **union by rank**, both operations run in almost $O(1)$ time.

Signal words: "connected components", "redundant connection", "accounts merge", "friend circles", "is there a cycle in an undirected graph".

19.2 Java Implementation

```
class UnionFind {
    private int[] parent;
    private int[] rank;

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;    // each node is its own parent initially
    }

    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);    // path compression
        }
        return parent[x];
    }

    public boolean union(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return false;    // already in the same set (would form a cycle)

        if (rank[rootX] < rank[rootY]) {
            int temp = rootX; rootX = rootY; rootY = temp;
        }
        parent[rootY] = rootX;    // union by rank
        if (rank[rootX] == rank[rootY]) rank[rootX]++;
        return true;
    }
}
```

19.3 Step-by-Step Visual Trace

```
union(0,1): parent=[1,1,2,3,4]    (0's root now points to 1)
              union(2,3): parent=[1,1,3,3,4]
union(1,2): find(1)=1, find(2)=3    parent=[1,1,3,1,4]    (attach root 3 under root 1)

Now find(0): 0->1->1 (path compressed) all point directly to root 1
Sets: {0,1,2,3} and {4}
```

19.4 Java Implementation — Number of Connected Components

```
public static int countComponents(int n, int[][] edges) {
    UnionFind uf = new UnionFind(n);
    int components = n;
    for (int[] edge : edges) {
        if (uf.union(edge[0], edge[1])) {
            components--;    // successful union means we merged two separate groups
        }
    }
    return components;
}
```

19.5 Complexity

Time: $O(\alpha(n))$ per operation (inverse Ackermann) — effectively constant time in practice.

Space: $O(n)$ for the parent and rank arrays.

19.6 Practice Problems (Easy → Hard)

Easy: Number of Provinces

Medium: Number of Connected Components · Redundant Connection · Accounts Merge · Graph Valid Tree

Hard: Number of Islands II · Swim in Rising Water · Smallest String With Swaps

20. Final Interview Playbook & Practice Roadmap

20.1 The Pattern Recognition Cheat Sheet

If you see this in the problem...	...think of this pattern
Sorted array, pair/triplet sum	Two Pointers
Contiguous subarray/substring with a condition	Sliding Window
Repeated range sum queries	Prefix Sum
Anagram, duplicate, grouping	Frequency Counting / Hashing
Grid, islands, flood fill	Matrix Traversal (BFS/DFS on grid)
Next greater/smaller element	Monotonic Stack
Reverse / cycle / middle of list	Linked List (fast-slow pointers)
Tree structure, ancestors, paths	Binary Tree Traversal (DFS/BFS)
Sorted dynamic data, kth smallest	Binary Search Tree
Kth largest/smallest, top K	Heap (PriorityQueue)
Shortest path, unweighted graph	BFS
All combinations/permutations	Backtracking
"maximize/minimize" with local choice works	Greedy
Count ways, min/max cost, overlapping subproblems	Dynamic Programming
Merge/insert/count overlapping ranges	Overlapping Intervals
Minimize the maximum / feasibility check	Binary Search on Answer
Prefix-based word lookup	Trie
XOR tricks, "without arithmetic operators"	Bit Manipulation
Grouping/connectivity, cycle in undirected graph	Union-Find

20.2 A 6-Week Practice Roadmap

Week 1: Java fundamentals, Arrays & Strings, Two Pointers, Sliding Window, Frequency Counting/Hashing.

Week 2: Prefix Sum, Stack & Monotonic Stack, Linked Lists, Matrix Traversal.

Week 3: Binary Trees & all traversals, Binary Search Trees, Binary Search & Binary Search on Answer.

Week 4: Heaps/Top-K, Graphs (BFS/DFS), Union-Find, Trie.

Week 5: Backtracking, Greedy, Overlapping Intervals.

Week 6: Dynamic Programming (all sub-categories) — give it real time. Mix in mock interviews and revisit weak areas.

20.3 During the Actual Interview

- 1. Clarify first.** Ask about input size, duplicates, edge cases before coding.
- 2. State brute force.** Say the $O(n^2)$ approach out loud, then say what pattern will optimize it.
- 3. Talk while you code.** Silence makes it hard for the interviewer to follow your reasoning.
- 4. Test with a small example.** Trace your code by hand before saying "I'm done."
- 5. State complexity at the end.** "This is $O(n)$ time, $O(1)$ space" shows efficiency-mindedness.

20.4 Java-Specific Interview Reminders

- Always use `.equals()` for String/object comparison, never `==`.
- Remember `ArrayList` needs wrapper types (`Integer` , not `int`).
- Override both `equals()` and `hashCode()` together, or your objects will misbehave in HashMaps/HashSets.
- Prefer `ArrayDeque` over the legacy `Stack` class for stack operations — it's faster and unsynchronized.
- Watch for integer overflow: use `long` when sums could exceed ~2.1 billion.
- Declare variables with interface types (`List<Integer> list = new ArrayList<>();`) — a good practice interviewers notice.

Final thought: Every hard problem is just 2-3 easy patterns combined. "Minimum Window Substring" = Sliding Window + Frequency Counting. "Word Search II" = Backtracking + Trie. Once patterns become second nature, even unfamiliar problems become a puzzle of "which patterns fit together here?" Good luck!